

PROGETTO MAADB — LAB 2025/2026

FinDetect: polyglot persistence Neo4j + MongoDB
sul benchmark LDBC FinBench

Pietro Sardi (matr. 989803) Pietro Longo (matr. 1062127)

Università degli Studi di Torino

A.A. 2025–2026

Sommario

FinDetect è un prototipo di applicazione web per l'interrogazione e l'analisi del dataset finanziario *LDBC FinBench*. Gli stessi dati sono caricati su due basi di dati non relazionali — un database a grafo (Neo4j) e un database a documenti (MongoDB) — per sfruttare ciascun motore dove rende di più: il grafo per le interrogazioni sui cammini fra conti, il documento per le schede anagrafiche aggregate. L'app espone sei query parametriche (lookup e analitiche, alcune cross-database) tramite un'interfaccia Flask. La relazione discute le scelte progettuali, le collega ai concetti del corso e analizza in modo critico i tempi di esecuzione misurati sui dati reali.

Indice

1	Introduzione	3
2	Accenni teorici	3
2.1	Modello a grafo e modello a documenti	3
2.2	Centralità di grado per la <i>hub detection</i>	4
2.3	Selettività e costo delle query	4
2.4	Aspetti distribuiti: replica, consistenza, partizionamento	4
3	Dataset e preprocessing	5
3.1	Il modello dei dati FinBench	5
3.2	LDBC FinBench e lo <i>scale factor</i>	5
3.3	Volumi caricati (dati reali)	6
3.4	ETL: dal CSV ai due database	6
4	Architettura del sistema	7
4.1	Struttura del codice	7
4.2	Le sei query e la loro classificazione	7
4.3	Interfaccia web	8
5	Risultati e analisi critica	9
6	Limitazioni	10
7	Conclusioni	10
8	Comandi per l'esecuzione	10

1 Introduzione

Il problema che affrontiamo è l'analisi di dati finanziari per scopi di *antifrode*: dato un grande insieme di conti, persone, aziende e transazioni, vogliamo rispondere a domande sia puntuali (“mostrami il profilo di questa persona”) sia analitiche (“trova i conti che fanno da snodo a tantissime transazioni”, “trova i giri di denaro che tornano al mittente”).

Il dato di partenza è il benchmark **LDBC FinBench**, uno standard pensato proprio per stressare i sistemi di gestione dati su carichi di lavoro di tipo finanziario. La traccia del progetto chiede di caricare gli stessi dati su due sistemi non relazionali e di costruire una web-app che permetta query parametriche di tipo *lookup* e query *analitiche*, di cui almeno una per categoria deve essere cross-database.

La scelta di non usare un singolo database relazionale nasce da un'osservazione semplice: il modello relazionale è ottimo quando i dati sono “piatti e omogenei”, ma diventa scomodo quando l'informazione interessante sta nelle *relazioni fra le entità*. Trovare un ciclo di pagamenti $A \rightarrow B \rightarrow C \rightarrow A$ in SQL richiederebbe più self-join annidati e costosi; su un grafo è una singola interrogazione di *raggiungibilità*. Da qui la scelta della *polyglot persistence*: usare due tecnologie diverse, ognuna nel suo dominio di forza.

Cosa fa l'applicazione. L'utente sceglie una delle sei query da un menù, imposta i parametri (con la possibilità di pescarne di realistici a caso dal dataset) e ottiene una tabella di risultati con il numero di righe e il tempo di esecuzione. Il prototipo è volutamente minimale, come richiesto dalla traccia: l'obiettivo non è un prodotto, ma una dimostrazione funzionante delle scelte di modellazione.

2 Accenni teorici

Questa sezione richiama i concetti che useremo per giustificare le scelte. Ogni nozione è spiegata in modo intuitivo e collegata a come è stata usata nel progetto.

2.1 Modello a grafo e modello a documenti

Un **database a grafo** rappresenta i dati come nodi (le entità) e archi (le relazioni), entrambi dotati di proprietà. Ha senso sceglierlo dove la natura dei dati è già una rete di relazioni, come nel nostro caso. La forza del modello è che abilita le **query di raggiungibilità**, cioè interrogazioni che chiedono se e come due nodi sono collegati da un cammino, che nel relazionale sono molto costose da esprimere e da eseguire. È esattamente il caso dei nostri cicli di transazioni (Q4) e della ricerca degli snodi (Q6).

Un **database a documenti** memorizza invece oggetti autodescrittivi (in MongoDB, documenti BSON simili a JSON). Si rinuncia alla rigidità dello schema relazionale in cambio di flessibilità: un documento può contenere direttamente le sue sotto-strutture annidate. Questo è ideale per la *scheda di una persona*, dove vogliamo anagrafica, conti e prestiti già raccolti insieme senza dover ricomporre tutto con dei join (Q2).

Perché due database e non uno. La *polyglot persistence* è la pratica di usare più tecnologie di memorizzazione nello stesso sistema, ciascuna per il tipo di accesso a cui è più adatta. Il costo è la **ridondanza controllata**: alcune informazioni (es. i transfer) vivono sia come archi in Neo4j sia come documenti in MongoDB. Lo accettiamo consapevolmente perché i due motori servono interrogazioni diverse: il grafo per i cammini, il documento per le aggregazioni anagrafiche. È un classico esempio di scelta di ingegneria che baratta un po' di spazio e di coerenza in cambio di prestazioni mirate.

Perché proprio Neo4j e MongoDB. Fra i database a grafo abbiamo scelto **Neo4j** perché è il più maturo e diffuso, ha un'edizione Community gratuita e usa *Cypher*, un linguaggio dichiarativo molto leggibile (un pattern come $A \rightarrow B \rightarrow C \rightarrow A$ si scrive quasi come si disegnerebbe su carta). Alternative come ArangoDB o Amazon Neptune sarebbero state valide ma meno immediate da installare e usare in locale. Fra i document store abbiamo scelto **MongoDB** per la stessa ragione di diffusione, per la potenza della *aggregation pipeline* e per la semplicità con cui si configura un replica set; un'alternativa come PostgreSQL con campi JSONB sarebbe stata di fatto un ritorno al relazionale, fuori dallo spirito “due sistemi non relazionali” richiesto dalla traccia.

2.2 Centralità di grado per la *hub detection*

La query Q6 cerca gli “snodi” della rete, cioè i conti che partecipano a un numero anomalo di transazioni. La misura naturale è la **centralità di grado** (*degree centrality*): il grado di un nodo è il numero di archi **TRANSFER** che lo toccano, contando sia le transazioni entranti sia quelle uscenti. Un conto con grado molto alto è un candidato hub: nello scenario antifrode è un punto che merita attenzione. Q6 calcola questo grado *dentro una finestra temporale* e offre due criteri per decidere chi è hub:

- **soglia assoluta** (parametro `minDegree`, il default): tiene i conti con grado maggiore o uguale alla soglia. È il criterio *operativo*, più vicino al caso d'uso antifrode/vigilanza dichiarato da LDBC FinBench: si vuole segnalare chi supera una certa attività indipendentemente dal resto del sistema;
- **top $x\%$** (parametro opzionale `topPct`): tiene la frazione più attiva della distribuzione. È un criterio *relativo*, adatto ad esplorare come sono distribuiti i gradi; per calcolarlo serve però l'intera classifica dei conti attivi, quindi costa un po' di più.

In entrambi i casi la query riporta, oltre ai primi 20 hub, quanti conti soddisfano il criterio e il grado minimo fra loro: così i due approcci si confrontano direttamente (ad esempio, a SF 1 sul triennio 2020–2022 il top 1% corrisponde a circa 2.600 conti con grado ≥ 72 , mentre la soglia 500 ne isola poche decine).

2.3 Selettività e costo delle query

Per ragionare sui tempi di esecuzione useremo il concetto di **selettività** di una condizione: la frazione dei dati in ingresso che supera il filtro. Un filtro molto selettivo lascia passare poche tuple, uno poco selettivo costringe il motore a toccare gran parte dei dati. Le nostre query di lookup sono molto selettive (filtrano su un singolo `id` indicizzato) e infatti costano pochi millisecondi; le analitiche a grafo come Q4 e Q6 sono poco selettive (devono esplorare molti archi) e questo si rifletterà nei tempi misurati (Sezione 5).

2.4 Aspetti distribuiti: replica, consistenza, partizionamento

La traccia chiede di collegare il progetto, dove pertinente, ai concetti di sistema distribuito (replica, partizionamento/sharding, consistenza, batch e stream processing). Va distinta la “consistenza” transazionale di un DBMS centralizzato (la C di ACID: ogni transazione porta la base di dati da uno stato valido a un altro) dalla consistenza di un sistema *distribuito*, che riguarda invece quanto le copie di un dato su nodi diversi sono allineate fra loro. Per gli aspetti distribuiti ci appoggiamo alla documentazione ufficiale dei sistemi usati, collegandoli a come il prototipo è effettivamente configurato.

Replica (MongoDB replica set). Nel nostro `docker-compose.yml` MongoDB è avviato in modalità *replica set* (`--replSet rs0`), con uno script di inizializzazione che chiama

`rs.initiate()`. Un replica set è un gruppo di nodi che mantengono la stessa copia dei dati con architettura *primary-secondary*: le scritture vanno al primario e vengono propagate ai secondari tramite un log delle operazioni (*oplog*). Nel prototipo il set ha un solo nodo — sufficiente in laboratorio — ma l'architettura è quella replicata. **Perché averlo, anche con un nodo?** Perché MongoDB abilita le transazioni multi-documento (con le loro garanzie ACID) solo sopra un replica set: il set è quindi un prerequisito architetturale, non un dettaglio.

Consistenza. Sopra un replica set MongoDB offre una consistenza *regolabile* tramite `write concern` (quanti nodi devono confermare una scrittura) e `read concern` (quanto recente deve essere la lettura). Con un solo nodo la lettura è di fatto fortemente consistente; in un set a più nodi si potrebbe scegliere fra letture più veloci ma potenzialmente indietro nel tempo (dai secondari) e letture più sicure (dal primario). È il tipico compromesso fra latenza e consistenza dei sistemi distribuiti, assente in un DBMS puramente centralizzato.

Partizionamento/sharding. Non lo abbiamo usato: alla scala del nostro dataset (*scale factor* 1, descritto sotto) i dati stanno comodamente su un singolo nodo, e introdurre lo sharding sarebbe stato **over-engineering**. Lo sharding diventa necessario quando i dati non entrano più su una macchina: MongoDB partiziona orizzontalmente le collezioni in base a una *shard key*, Neo4j Community (quello che usiamo) non supporta il clustering. Lo citiamo come scelta consapevolmente scartata per la scala in gioco.

Batch vs stream processing. Il dataset è prodotto in **batch**: il datagen ufficiale LDBC gira su *Apache Spark*, il motore di elaborazione distribuita per eccellenza, e genera i CSV in una sola passata. Anche il nostro ETL è batch (caricamento massivo dei CSV nei due database). Non usiamo *stream processing* perché i dati sono statici, generati una volta sola; un sistema antifrode reale, che deve reagire alle transazioni mentre accadono, userebbe invece un motore a flussi (es. Kafka + Spark Streaming) con elaborazione a finestre temporali. Lo segnaliamo come naturale estensione.

3 Dataset e preprocessing

3.1 Il modello dei dati FinBench

Il dataset descrive un piccolo mondo finanziario fatto di poche entità e di una fitta rete di relazioni fra loro. Le entità (i nodi in Neo4j, le collezioni in MongoDB) sono:

- **Person e Company:** i titolari, persone fisiche e aziende;
- **Account:** i conti, ciascuno con un tipo (es. *retirement account*, *credit card*) e un intestatario;
- **Loan:** i prestiti, con importo, saldo e finalità;
- **Medium:** gli strumenti di pagamento usati nelle operazioni.

Il cuore del dataset sono però le *relazioni*: una persona *possiede* conti (`personOwnAccount`) e *richiede* prestiti (`personApplyLoan`); soprattutto, i conti si scambiano denaro con i `TRANSFER` — il tipo di arco più importante per noi, oltre 1,3 milioni di istanze — e fanno operazioni di `DEPOSIT`, `WITHDRAW` e `REPAY`. È proprio questa rete di transazioni fra conti a rendere naturale il modello a grafo: le nostre query analitiche (cicli, hub) sono domande *sulla forma della rete*, non sui singoli record.

3.2 LDBC FinBench e lo *scale factor*

Il dataset è generato dal datagen ufficiale LDBC FinBench (tag `v0.1.0`) con *scale factor* (SF) pari a 1. Lo *scale factor* è il “rubinetto” che regola la dimensione dei dati generati: SF 1

produce un dataset abbastanza grande da essere interessante (milioni di archi) ma abbastanza piccolo da girare su un portatile.

Perché SF 1 e non di più. La generazione è l’operazione più pesante dell’intero progetto, perché Spark deve simulare l’attività finanziaria. Su un portatile Apple Silicon la generazione di SF 1 ha richiesto circa 7–8 minuti, limitando la memoria a 8 GB. SF più alti (10, 100) avrebbero moltiplicato i tempi e lo spazio senza aggiungere nulla alla dimostrazione delle scelte di modellazione, che è l’obiettivo del progetto. È la stessa logica con cui in un esperimento si sceglie un campione rappresentativo invece dell’intera popolazione.

3.3 Volumi caricati (dati reali)

La Tabella 1 e la Tabella 2 riportano i conteggi *effettivamente misurati* sui due database dopo l’ETL. In Neo4j il dataset è fatto di **643.241 nodi** e **5.372.701 archi**.

		Arco (tipo)	Conteggio
Nodo (label)	Conteggio	WITHDRAW	2.011.359
Account	264.075	TRANSFER	1.379.527
Loan	159.166	DEPOSIT	512.680
Medium	100.000	SIGNIN	451.362
Person	80.000	OWN	264.075
Company	40.000	REPAY	262.571
Totale nodi	643.241	INVEST	260.156
		APPLY	159.166
		GUARANTEE	71.805
		Totale archi	5.372.701

Tabella 1: Volumi in Neo4j: nodi per label e archi per tipo.

Collezione	Documenti	Collezione	Documenti
account	264.075	person	80.000
transfer	1.379.527	company	40.000
loan	159.166	personOwnAccount	175.956
medium	100.000	companyOwnAccount	88.119
personApplyLoan	106.346		

Tabella 2: Volumi in MongoDB (database `findetect`): documenti per collezione.

Si noti la coerenza fra i due database: i 264.075 conti sono sia nodi `Account` sia documenti `account`, e gli 1.379.527 transfer sono sia archi `TRANSFER` sia documenti `transfer`. Anche le due collezioni-link `personOwnAccount` e `companyOwnAccount` sommano esattamente a 264.075: in FinBench ogni conto ha un solo intestatario, persona o azienda, ed è per questo che le query cross-database (Q3, Q6) interrogano entrambe. È la ridondanza controllata di cui sopra: la stessa entità vive nei due mondi con la forma più comoda per ciascuno.

3.4 ETL: dal CSV ai due database

Il caricamento è gestito da due script Python (`etl/load_neo4j.py` e `etl/load_mongo.py`) che leggono i CSV grezzi prodotti dal datagen e li inseriscono nei rispettivi database. Le scelte principali:

- **Caricamento a blocchi (batch).** Gli inserimenti sono raggruppati in lotti invece di una riga alla volta: scrivere milioni di record uno per uno sarebbe lentissimo per via del costo fisso di ogni operazione di I/O.
- **Indici prima dei dati.** In Neo4j creiamo un indice su `Account.id` prima di caricare gli archi, così la ricerca dei nodi a cui agganciare ogni `TRANSFER` non degenera in una scansione completa. In MongoDB l'indice su `_id` è automatico.
- **Le date come interi.** I timestamp sono salvati come millisecondi dall'epoch (interi), più compatti e veloci da confrontare delle stringhe.

4 Architettura del sistema

4.1 Struttura del codice

Il progetto è organizzato in moduli piccoli e con un compito chiaro:

- `docker-compose.yml` — avvia Neo4j e MongoDB in container;
- `datagen/run_datagen.sh` — clona, compila e lancia il datagen Spark;
- `etl/` — gli script di caricamento CSV verso i due database;
- `app/db/` — due moduli (`mongo.py`, `neo4j.py`) che incapsulano la connessione: il resto del codice chiede una connessione e non sa nulla di come viene creata;
- `app/queries/` — un file per ognuna delle sei query, più un modulo `_common.py` con le funzioni condivise (conversione date, misura dei tempi);
- `app/app.py` — l'applicazione Flask con le rotte.

Perché separare connessione e query. Ogni modulo in `app/db/` espone una sola funzione (`get_driver()`, `get_db()`) che restituisce una connessione riusabile. È una forma leggera di *repository pattern*: se domani cambiassimo le credenziali o spostassimo un database, toccheremmo un solo file invece di tutte le query.

4.2 Le sei query e la loro classificazione

La traccia chiede almeno due query di lookup (di cui una cross-database) e almeno due analitiche (di cui una cross-database). La Tabella 3 mostra come le nostre sei query coprono e superano il requisito.

ID	Tipo	Database	Descrizione
Q1	lookup	Neo4j	Transfer uscenti da un conto in una finestra temporale
Q2	lookup	MongoDB	Profilo completo di una persona (anagrafica + conti + prestiti)
Q3	lookup	Cross-DB	Scheda conto + intestatario (Mongo), top-10 vicini (Neo4j)
Q4	analitica	Neo4j	Cicli di transfer $A \rightarrow B \rightarrow C \rightarrow A$ sopra soglia
Q5	analitica	MongoDB	Top-10 conti per volume trasferito in un mese
Q6	analitica	Cross-DB	Hub per grado, soglia o top- $x\%$ (Neo4j) + intestatario (Mongo)

Tabella 3: Le sei query: tre di lookup e tre analitiche, due delle quali cross-database.

Esempio di query a grafo (Q4, cicli). La forza del modello a grafo si vede nella query dei cicli: un pattern che in SQL sarebbe un incubo di self-join, in Cypher è una sola riga di *pattern matching*.

```

1 MATCH (a:Account)-[t1:TRANSFER]->(b:Account)
2   -[t2:TRANSFER]->(c:Account)-[t3:TRANSFER]->(a)
3 WHERE t1.amount >= $minAmount AND t2.amount >= $minAmount
4   AND t3.amount >= $minAmount

```

```

5 AND t1.timestamp < t2.timestamp
6 AND t2.timestamp < t3.timestamp
7 AND t3.timestamp - t1.timestamp <= $windowMs
8 RETURN a.id, b.id, c.id, t1.amount, t2.amount, t3.amount
9 LIMIT 50

```

Listing 1: Q4 – ricerca di cicli a tre passi in Cypher.

Esempio di query cross-database (Q6). Q6 fa lavorare i due motori in sequenza: prima Neo4j trova gli hub (i conti con grado alto), poi MongoDB arricchisce ciascun hub con tipo di conto, data di apertura e profilo dell’intestatario (persona o azienda). È il pattern “il grafo trova *chi*, il documento dice *cosa*”.

```

1 # 1) Neo4j: trova gli hub per grado dentro la finestra
2 # (CYPHER_THRESHOLD se si usa la soglia, CYPHER_TOP_PCT se si usa il top x%)
3 hubs = session.run(CYPHER_THRESHOLD, startMs=..., endMs=..., minDegree=...).data()
4 ids = [h["accountId"] for h in hubs]
5
6 # 2) MongoDB: metadati del conto + intestatario, con una find($in) per collezione
7 meta = {d["_id"]: d for d in db.account.find({"_id": {"$in": ids}})}
8 owners = owners_of(ids) # legge personOwnAccount e companyOwnAccount

```

Listing 2: Q6 – la parte di orchestrazione cross-database (Python).

L’helper `owners_of` (in `app/queries/_common.py`) è condiviso con Q3: risolve l’intestatario passando dalle due collezioni-link (`personOwnAccount`, `companyOwnAccount`) alle rispettive anagrafiche, sempre con query `$in` in blocco e mai una query per conto.

Esempio di aggregazione (Q5, MongoDB). Dove Neo4j usa il pattern matching, MongoDB usa la *aggregation pipeline*: una sequenza di stadi che trasformano i documenti a catena. Q5 trova i dieci conti con più volume trasferito in un mese.

```

1 pipeline = [
2     {"$match": {"timestamp": {"$gte": start_ms, "$lt": end_ms}}},
3     {"$group": {"_id": "$accountSrc",
4                 "totalAmount": {"$sum": "$amount"},
5                 "numTransfers": {"$sum": 1}}},
6     {"$sort": {"totalAmount": -1}},
7     {"$limit": 10},
8     {"$lookup": {"from": "account", "localField": "_id",
9                 "foreignField": "_id", "as": "account"}}},
10 ]

```

Listing 3: Q5 – aggregation pipeline di MongoDB.

Si legge come una catena di trasformazioni: `$match` filtra il mese (lo stadio selettivo che rende la query veloce), `$group` somma gli importi per conto, `$sort` e `$limit` tengono i primi dieci, `$lookup` aggancia i metadati del conto. È l’equivalente document-oriented di un `GROUP BY` con `join`. Avere i due paradigmi a confronto — pattern matching dichiarativo su grafo e pipeline procedurale su documenti — è uno dei punti più istruttivi del progetto.

4.3 Interfaccia web

L’interfaccia (Flask + Tailwind) presenta le sei query come schede selezionabili; per ogni query mostra un form con i parametri. Una funzione di comodo (`/api/random`) pesca valori *realmente presenti* nel dataset, così chi prova l’app non deve conoscere a memoria gli identificativi dei conti (che sono interi a 19 cifre). Questa scelta nasce da un problema concreto di usabilità: senza, l’utente non saprebbe cosa scrivere nei campi e otterrebbe sempre zero risultati.

5 Risultati e analisi critica

Abbiamo misurato ogni query eseguendola cinque volte di fila con parametri di default realistici. La Tabella 4 riporta il numero di righe restituite e il tempo *a regime* (la mediana delle esecuzioni), insieme al tempo della *prima* esecuzione, sempre più alto.

ID	Tipo	DB	Righe	A regime (ms)	Prima run (ms)
Q1	lookup	Neo4j	100	10,0	169,7
Q2	lookup	MongoDB	1	0,5	63,9
Q3	lookup	Cross-DB	1*	6,2	67,4
Q4	analitica	Neo4j	50	660,8	906,9
Q5	analitica	MongoDB	10	1,0	35,9
Q6	analitica	Cross-DB	20	1289,6	1915,1

Tabella 4: Prestazioni misurate (5 esecuzioni). *Q3 restituisce un documento che contiene a sua volta i 10 conti vicini.

I numeri raccontano una storia molto netta e coerente con la teoria della Sezione 2.

Q1 — Transfer uscenti (lookup, Neo4j). 100 righe in circa 10 ms. La query è molto selettiva: parte da un singolo conto (indice su `Account.id`) e segue solo i suoi archi uscenti dentro la finestra. È il caso ideale per il grafo: navigazione locale a partire da un nodo noto.

Q2 — Profilo persona (lookup, MongoDB). Un solo documento in mezzo millisecondo, la query più veloce in assoluto. La persona è trovata per `_id` (indice automatico) e il documento contiene già, tramite quattro *lookup* a catena, i suoi conti e prestiti. È la dimostrazione del vantaggio del modello a documenti per le schede aggregate.

Q3 — Scheda conto + vicini (lookup, cross-DB). Circa 6 ms. Qui i due database collaborano: MongoDB dà i metadati del conto e dell'intestatario, Neo4j i dieci conti più connessi per numero di transfer. Il costo è ancora basso perché tutto parte da un singolo conto noto.

Q4 — Cicli a tre passi (analitica, Neo4j). Qui il tempo sale a circa 660 ms, due ordini di grandezza sopra le lookup. È atteso: la ricerca del pattern $A \rightarrow B \rightarrow C \rightarrow A$ è poco selettiva, perché il motore deve esplorare moltissimi cammini di lunghezza tre prima di scoprire quali si chiudono in ciclo. È il prezzo di un'interrogazione potente che sul relazionale sarebbe ancora più cara.

Q5 — Top-10 volume mensile (analitica, MongoDB). Circa 1 ms, sorprendentemente veloce per un'analitica. Il motivo è che l'aggregazione opera su un filtro temporale selettivo (un solo mese) e MongoDB la risolve con la sua pipeline nativa. Mostra che “analitica” non significa automaticamente “lenta”: conta quanto è selettivo il filtro a monte.

Q6 — Hub detection (analitica, cross-DB). La query più pesante, circa 1,3 secondi. Deve scorrere *tutti* i transfer dentro la finestra temporale, calcolare il grado di ogni conto e tenere solo quelli sopra soglia. È poco selettiva per costruzione — guarda l'intera rete — e per questo è la più costosa. La variante top- $x\%$ costa ancora un po' di più (circa 1,5 s a regime con $x = 1$): non basta più tenere i primi 20 in un *heap*, bisogna ordinare e materializzare tutta la classifica per sapere dove cade il taglio percentuale. Un dettaglio implementativo che ha

contato: raggruppare per *nodo* e leggere `a.id` solo sui conti sopravvissuti al filtro, invece che su tutti i 2,7 milioni di estremi di arco, vale circa 0,7 s.

Il costo della “prima volta”. In tutte le query la prima esecuzione è nettamente più lenta delle successive (per Q2 si passa da 64 ms a mezzo millisecondo). È l’effetto del *caching*: alla prima esecuzione i dati non sono ancora nella memoria centrale dei database e vanno letti da disco; dopo, restano nel buffer e le query a regime sono molto più veloci. È il principio di *località temporale* su cui si basa la gestione del buffer: un dato appena letto ha alta probabilità di essere riletto a breve, quindi tenerlo in memoria ammortizza il costo.

Lettura d’insieme. Le quattro lookup/analitiche selettive (Q1, Q2, Q3, Q5) stanno tutte sotto i 10 ms; le due analitiche di grafo poco selettive (Q4, Q6) costano centinaia/migliaia di ms. La discriminante non è “grafo contro documento” né “lookup contro analitica”, ma la **selettività**: più dati la query è costretta a toccare, più tempo impiega. È il principio di base dell’ottimizzazione delle interrogazioni.

6 Limitazioni

- **Ridondanza e coerenza.** Tenere i transfer in due database significa che un aggiornamento andrebbe propagato a entrambi. Nel prototipo i dati sono in sola lettura, quindi il problema non si pone; in un sistema reale servirebbe un meccanismo di sincronizzazione (es. *change data capture*).
- **Scala singola.** Tutto gira su un nodo per database. Q6, che scorre l’intera rete, a scale factor molto più alti diventerebbe proibitiva e richiederebbe sharding o pre-calcolo dei gradi (materializzazione).
- **Hub detection: due criteri, nessuno “giusto”.** Q6 offre soglia assoluta e top- $x\%$, ma la scelta della soglia (o della percentuale) resta a carico dell’analista: un criterio più robusto userebbe statistiche della distribuzione (es. media più k deviazioni standard, o percentili calcolati per periodo).
- **Nessuna elaborazione a flusso.** Il sistema lavora su dati statici. Un caso antifrode reale, che deve reagire alle transazioni in tempo reale, richiederebbe stream processing (Sezione 2.4).

7 Conclusioni

FinDetect mostra in pratica perché la polyglot persistence è una scelta sensata su dati finanziari: le interrogazioni sui cammini (cicli, hub) vivono naturalmente nel grafo, le schede aggregate nel documento, e ogni motore lavora dove rende di più. L’analisi dei tempi conferma quanto anticipato nella Sezione 2: il costo di una query è governato dalla sua selettività, non dal tipo di database. Le sei query coprono e superano il requisito della traccia (lookup e analitiche, con casi cross-database), e l’interfaccia rende il tutto dimostrabile senza conoscere a memoria gli identificativi del dataset. I limiti principali — ridondanza, scala singola, assenza di stream — sono noti e indicano le naturali direzioni di sviluppo.

8 Comandi per l’esecuzione

```
1 # 1) database in Docker (Neo4j + MongoDB replica set)
2 docker compose up -d
3
4 # 2) generazione del dataset SF 1 (una volta sola, ~8 min)
```

```
5 ./datagen/run_datagen.sh
6
7 # 3) ambiente Python
8 python3.11 -m venv .venv
9 source .venv/bin/activate
10 pip install -r requirements.txt
11
12 # 4) caricamento dati nei due database
13 python -m etl.load_mongo
14 python -m etl.load_neo4j
15
16 # 5) avvio dell'app web -> http://localhost:5050
17 python -m app.app
18
19 # 6) (opzionale) smoke test: le 6 query coi parametri di default
20 python test_smoke.py
```

Listing 4: Avvio completo del prototipo.